Nama : Elsadanika Maura Sitorus (211344013)

Praktikum JAVA 3

A. Tujuan Umum :

1. Menggunakan *keyword* static pada program yang dibuat
2. Menggunakan *method* pada program yang dibuat
3. Menggunakan *overloading methods* pada program yang dibuat

B. Menggunakan **Tujuan Khusus :**

1. Mampu menjelaskan dan mengimplementasikan fungsi *keywords static* pada suatu variabel maupun *method* di dalam suatu *class*
2. Mampu menjelaskan dan mengimplementasikan suatu *method* selain *main method* pada suatu *class* (menciptakan dan memanggil suatu *method* di dalam *class*)
3. Mampu membedakan fungsi *constructor* dengan fungsi *method* pada suatu *class* yang dibuat
4. Mampu menjelaskan dan mengimplementasikan konsep *overloading* pada suatu *method* yang berkaitan dengan penamaam dan pemberian parameter *method* tersebut di dalam *class*
5. Mampu mengimplementasikan fungsi *return* dan *non-return* pada suatu *method* yang dibuat secara *overloading*

C. Instruksi dan Permasalahan

1. Buatlah file program JAVA dengan nama VariabelStatic.java.
   a. Buatlah nama public class sesuai dengan nama file JAVA
   b. Tambahkan variabel pada suatu *class* dengan nama hitung dan memiliki tipe data integer
   c. Tambahkan main *method* di dalam lingkup public class
   d. Buatlah program untuk mengisi variabel data dengan bilangan tertentu dan menampilkan ke layar monitor.
   e. Lakukan dengan pembuktian apakah program tersebut dapat di-*compile*
   f. Pindahkan posisi variabel dari lingkup *class* ke lingkup *main method*
   g. Lakukan dengan pembuktian apakah program tersebut dapat di-*compile*
   h. Lakukan langkah b. dan tambahkan *keyword static* di depan tipe data
   i. Lakukan dengan pembuktian apakah program tersebut dapat di-*compile*
   j. Lakukan pengisian variabel di dalam lingkup *main method* dengan instruksi berikut :

```
VariabelStatic.data = 10;
```

k. Lakukan pemanggilan variabel untuk dicetak di layar monitor seperti instruksi berikut :

```
System.out.print("nilai variabel data  =  "+VariabelStatic.data);
```

l. Jelaskan percobaan tersebut berkaitan dengan *static* variabel di dalam *class*

2. Buatlah file program JAVA dengan nama StaticMethod.java
   a. Buatlah nama public class sesuai dengan nama file JAVA
   b. Tambahkan main *method* di dalam lingkup public class dan tambahkan instruksi berikut di dalamnya :

```
Data2( );
```

   c. Buatlah di dalam lingkup *class* suatu *method* dengan nama data2 seperti berikut ini :

```
public void data2() {
        System.out.print("Hello students…!")
}
```

   d. Lakukan dengan pembuktian, apakah program tersebut dapat di-*compile* atau tidak
   e. Tambahkan *keyword* static sebelum *modifier void* pada *method* data2( ) dan lakukan dengan pembuktian mengenai pengaruh yang terjadi pada program tersebut
   f. Jelaskan percobaan tersebut berkaitan dengan *static method* di dalam *class*

3. Buatlah file program JAVA dengan nama ProgramMethod2.java.
   a. Buatlah nama public class sesuai dengan nama file JAVA
   b. Tambahkan main *method* di dalam lingkup class ProgramMethod2
   c. Buatlah class lain di dalam file ProgramMethod2.java dengan nama class Method2
   d. Tambahkan method di dalam *class Method2* dengan nama cetak ( ) dan ber-tipe void serta mengandung *body of method* berikut :

```
System.out.print("memanggil method yang berada di kelas lain");
```

   e. Lakukan pemanggilan *cetak( )* melalui *main method* dengan instruksi berikut :

```
Cetak ( );
```

   f. Tentukan dengan pembuktian apakah program dapat di-*compile*
   g. Gantikan langkah e. dengan instruksi berikut :

```
method2 objek = new method2( );
objek.cetak ( );
```

h. Tentukan hasil *output* kode program di atas dan jelaskan hasil pembuktian dalam bentuk.

4. Buatlah file program JAVA dengan nama Method3.java
   a. Buatlah nama public class sesuai dengan nama file JAVA
   b. Tambahkan main method di dalam lingkup *class* Method3
   c. Buatlah class lain di dalam file Method3.java dengan nama class MethodLain
   d. Tambahkan method di dalam *class MethodLain* dengan nama cetak ( ) ber-tipe void dan tambahkan *static modifier* di depan *keyword void* serta sisipkan pada *body of method*, instruksi berikut :

```
System.out.print("memanggil static method yang berada di kelas lain");
```

   e. Tambahkan instruksi berikut di dalam lingkup *main method* :

```
method2 objek = new method2( );
objek.cetak ( );
```

   f. Tentukan hasil output kode program pada langkah e.
   g. Gantikan langkah e. dengan instruksi berikut :

```
method2.cetak ( );
```

   h. Tentukan hasil output kode program pada langkah g.
   i. Jelaskan dengan pembuktian apakah terdapat perbedaan *output* pada langkah e. dan g.

5. Buatlah file program JAVA dengan nama OvMethod.java
   a. Buatlah nama public class sesuai dengan nama file JAVA
   b. Buatlah main method di dalam lingkup class OvMethod
   c. Sisipkan instruksi berikut di dalam lingkup *main method* :

```
cetak (25 );
cetak ("hallo Java..");
```

   d. Buatlah suatu *method* di dalam lingkup *class* yang sama dan diberi nama cetak ( ) dengan tipe *void* dan *static*, kemudian memiliki satu parameter bertipe integer, serta memiliki *body of method* berikut :

```
System.out.println(" nilai y adalah " +y);
```

e. Buatlah kembali suatu *method* dengan nama cetak ( ) dengan tipe *void* dan *static*, kemudian memiliki satu parameter bertipe String, serta memiliki *body of method* berikut :

```
System.out.println(“ kami ucapkan “ +y);
```

f. Tentukan dengan pembuktian apakah program ini dapat di-*compile*

g. Jelaskan apakah yang membedakan kedua *method* cetak ( ) tersebut dan sebutkan pula apa yang menjadi kesamaannya

6. Buatlah file program JAVA dengan nama OvMethod2.java

a. Buatlah nama public class sesuai dengan nama file JAVA

b. Sisipkan kode program berikut pada lingkup *class*

```
public static void main(String[] args) {

    System.out.println("nilai y adalah " + cetak(25));
    System.out.println("nilai dari pemanggilan method cetak adalah "
            + cetak("Hallo Java2"));

}

static int cetak(int y) {
    System.out.println("nilai y adalah " + y);
    return y;
}

static int cetak(String x) {
    System.out.println("kami ucapkan " + x);
    int y = 10;
    return y;
}
```

c. Tentukan dengan pembuktian, apakah file OvMethod2.java dapat di-*compile*

d. Jelaskan menurut anda, apakah pemberian nama dan nilai return yang sama namun membedakan parameternya pada beberapa *method* yang dibuat merupakan salahsatu bentuk pembuatan *overloading method*

7. Buatlah file program JAVA dengan nama OvMethod3.java

a. Ubahlah kode program pada pertanyaan no.6 sehingga beberapa *method* dengan nama cetak ( ) tersebut dibedakan hanya pada nilai *return*-nya saja, namun tipe data parameter dan jumlah parameter serta nama *method* dibuat sama.

b. Jelaskan dengan pembuktian, apakah program tersebut dapat di-*compile* dan jika tidak dapat di-*compile*, mengapa hal ini merupakan cara yang “salah” dalam pembuatan *overloading method*

Nama : Elsadanika Maura Sitorus (211344013)

D. Hasil Praktikum

1) Program 1

- Poin e

VariabelStatic - Notepad

File Edit Format View Help

```
public class VariabelStatic{
        int data;
        public static void main(String[] args) {
                data = 50;
                System.out.println(data);
                //VariabelStatic.data = 10;
                //System.out.println("nilai variabel data = "+ VariabelStatic.data);
    }
}
```

```
D:\KULIAH SMT 4\APKOM\040423>javac VariabelStatic.java
VariabelStatic.java:4: error: non-static variable data cannot be referenced from a static context
                data = 50;
                ^
VariabelStatic.java:5: error: non-static variable data cannot be referenced from a static context
                System.out.println(data);
                                   ^
2 errors
```

- Poin g

VariabelStatic - Notepad

File Edit Format View Help

```
public class VariabelStatic{
        public static void main(String[] args) {
                int data;
                data = 50;
                System.out.println(data);
                //VariabelStatic.data = 10;
                //System.out.println("nilai variabel data = "+ VariabelStatic.data);
    }
}
```

```
D:\KULIAH SMT 4\APKOM\040423>javac VariabelStatic.java

D:\KULIAH SMT 4\APKOM\040423>java VariabelStatic
50
```

- Poin i

VariabelStatic - Notepad

File Edit Format View Help

```
public class VariabelStatic{
        static int data;
        public static void main(String[] args) {
                data = 50;
                System.out.println(data);
                //VariabelStatic.data = 10;
                //System.out.println("nilai variabel data = "+ VariabelStatic.data);
    }
}
```

Nama : Elsadanika Maura Sitorus (211344013)

```
D:\KULIAH SMT 4\APKOM\040423>javac VariabelStatic.java

D:\KULIAH SMT 4\APKOM\040423>java VariabelStatic
50
```

- Poin k

VariabelStatic - Notepad

File Edit Format View Help

```
public class VariabelStatic{
        static int data;
        public static void main(String[] args) {
        //int hitung;
                data = 50;
                System.out.println(data);
                VariabelStatic.data = 10;
                System.out.println("nilai variabel data = "+ VariabelStatic.data);
    }
}
```

```
D:\KULIAH SMT 4\APKOM\040423>javac VariabelStatic.java

D:\KULIAH SMT 4\APKOM\040423>java VariabelStatic
50
nilai variabel data = 10
```

2) Program 2

- Poin d

StaticMethod - Notepad

File Edit Format View Help

```
public class StaticMethod{
        public static void main(String[] args){
        data2();
        }
        public void data2(){
        System.out.print("Hello students...!");
        }
}
```

```
D:\KULIAH SMT 4\APKOM\040423>javac StaticMethod.java
StaticMethod.java:3: error: non-static method data2() cannot be referenced from a static context
        data2();
        ^
1 error
```

Nama : Elsadanika Maura Sitorus (211344013)

- Poin e

*StaticMethod - Notepad

File Edit Format View Help

```
public class StaticMethod{
        public static void main(String[] args){
        data2();
        }
        public static void data2(){
        System.out.print("Hello students...!");
        }
}
```

```
D:\KULIAH SMT 4\APKOM\040423>javac StaticMethod.java

D:\KULIAH SMT 4\APKOM\040423>java StaticMethod
Hello students...!
```

3) Program 3

- Poin f

ProgramMethod2 - Notepad

File Edit Format View Help

```
class ProgramMethod2{
        public static void main(String[] args){
        cetak();
        }
}
class Method2{
        public void cetak(){
        System.out.print("memanggil method yang berada di kelas lain");
        }
}
```

```
D:\KULIAH SMT 4\APKOM\040423>javac ProgramMethod2.java
ProgramMethod2.java:3: error: cannot find symbol
        cetak();
        ^
  symbol:   method cetak()
  location: class ProgramMethod2
1 error
```

- Poin h

```
class ProgramMethod2{
        public static void main(String[] args){
        Method2 objek = new Method2();
        objek.cetak();
        }
}
class Method2{
        public void cetak(){
        System.out.print("memanggil method yang berada di kelas lain");
        }
}
```

```
D:\KULIAH SMT 4\APKOM\040423>javac ProgramMethod2.java

D:\KULIAH SMT 4\APKOM\040423>java ProgramMethod2
memanggil method yang berada di kelas lain
```

4) Program 4

- Poin f

Method3 - Notepad

File Edit Format View Help

```
class Method3{
        public static void main(String[] args){
        Method2 objek = new Method2();
        objek.cetak();
        }
}
class MethodLain{
        public static void cetak(){
        System.out.print("memanggil static method yang berada di kelas lain");
        }
}
```

```
D:\KULIAH SMT 4\APKOM\040423>javac Method3.java

D:\KULIAH SMT 4\APKOM\040423>java Method3
memanggil method yang berada di kelas lain
D:\KULIAH SMT 4\APKOM\040423>
```

- Poin h

Method3 - Notepad

File Edit Format View Help

```
class Method3{
        public static void main(String[] args){
        Method2.cetak();
        }
}
class MethodLain{
        public static void cetak(){
        System.out.print("memanggil static method yang berada di kelas lain");
        }
}
```

Nama : Elsadanika Maura Sitorus (211344013)

```
D:\KULIAH SMT 4\APKOM\040423>javac Method3.java
Method3.java:3: error: non-static method cetak() cannot be referenced from a static context
        Method2.cetak();
                ^
1 error
```

5) Program 5

```
class OvMethod{
        public static void main(String[] args){
                cetak(25);
                cetak("hallo Java..");
        }
        public static void cetak(int y){
                System.out.println("nilai y adalah "+y);
        }
        public static void cetak(String y){
                System.out.println("kami ucapkan "+y);
        }
}
```

```
D:\KULIAH SMT 4\APKOM\040423>javac OvMethod.java

D:\KULIAH SMT 4\APKOM\040423>java OvMethod
nilai y adalah 25
kami ucapkan hallo Java..
```

6) Program 6

```
class OvMethod2{
        public static void main(String[] args){
                System.out.println("nilai y adalah " +cetak(25));
                System.out.println("nilai dari pemanggilan method cetak adalah " +cetak("Hallo Java2"));
        }
        static int cetak(int y){
                System.out.println("nilai y adalah " +y);
                return y;
        }
        static int cetak(String x){
                System.out.println("Kami ucapkan "+x);
                int y = 10;
                return y;
        }
}
```

```
D:\KULIAH SMT 4\APKOM\040423>javac OvMethod2.java

D:\KULIAH SMT 4\APKOM\040423>java OvMethod2
nilai y adalah 25
nilai y adalah 25
Kami ucapkan Hallo Java2
nilai dari pemanggilan method cetak adalah 10
```

7) Program 7

```
class OvMethod3{
        public static void main(String[] args){
                System.out.println("nilai y adalah " +cetak(25));
                System.out.println("nilai dari pemanggilan method cetak adalah " +cetak("Hallo Java2"));
        }
        static int cetak(int y){
                System.out.println("nilai y adalah " +y);
                return y;
        }
        static int cetak(String x){
                System.out.println("Kami ucapkan "+x);
                int y = 10;
                return x;
        }
}
```

```
D:\KULIAH SMT 4\APKOM\040423>javac OvMethod3.java
OvMethod3.java:13: error: incompatible types: String cannot be converted to int
                return x;
                       ^
1 error
```

E. Analisis

Melalui praktikum yang telah dilaksanakan, pada program pertama diketahui bahwa data variabel perlu disimpan dalam method yang berkaitan dengannya jika tanpa “static” didepan tipe variabel. Program dapat di-compile setelah variabel disimpan didalam method main. Akan tetapi, ketika digunakan tipe “static” untuk variabel diluar method, maka program dapat berjalan sempurna dan menghasilkan output yang sesuai dengan keinginan soal.

Lalu pada program kedua, jenis method yang dimiliki adalah static untuk main, dan non static untuk method data2. Untuk hal tersebut, diperlukan metode pemanggilan berjenis static-static sehingga method data2 tidak bisa dipanggil hanya dengan perintah `data2();` akan tetapi jika tipe method diubah menjadi static-static, maka program tersebut dapat dijalankan dengan sempurna.

Pada program ketiga, untuk menghasilkan output yang sesuai diperlukan perintah pemanggilan method static-nonstatic. Dengan begitu, perintah yang digunakan adalah `Method2 objek = new Method2();` lalu dilanjutkan dengan `objek.cetak();` sehingga program dapat dijalankan dan mengeluarkan output yang sesuai dengan keinginan soal.

Pada program keempat, class Method3 memanggil Method2, yaitu class yang dibuat sebelumya. Dengan begitu pemanggilan yang berlangsung adalah bertipe static-nonstatic. Program menghasilkan output yang sesuai dengan perintah soal. Sedangkan ketika perintah pemanggilan dilakukan dalam tipe static-static, program tidak bisa dijalankan.

Pada program lima, digunakan overloading. Overloading pada Java terjadi ketika terdapat beberapa method dengan nama yang sama di dalam satu class, namun memiliki tipe, jumlah, atau urutan parameter yang berbeda. Dengan kata lain, method yang di-overload mempunyai nama yang sama tetapi parameter yang berbeda. Overloading memungkinkan developer untuk membuat method dengan nama yang sama, tetapi dapat menerima tipe data atau parameter yang berbeda untuk mempermudah penggunaan program. Ketika sebuah method di panggil, compiler Java akan menentukan method yang harus dipanggil berdasarkan jumlah dan tipe parameter yang diberikan pada pemanggilan method tersebut.

Overloading yang berjalan memiliki variabel y sebagai utamanya. Method cetak pertama dengan tipe integer dan variable y merujuk kepada nilai 25, sedangkan pada method cetak kedua dengan tipe string merujuk kepada data “hallo java”. Hal ini adalah bentuk penyesuaian dari tipe data dengan jenis data yang dijabarkan, yaitu integer dengan integer (bilangan bulat) dan String dengan String (kalimat).

Pada program keenam, memiliki perbedaan dengan program lima yaitu pada penambahan variabel y yang bernilai 10 dengan jenis integer yang dideklarasikan dalam method cetak kedua. Hal ini menghasilkan output yang berbeda dengan program 5, yaitu ada nilai pemanggilan method cetak yaitu 10. Program ketujuh membuktikan bahwa nilai return yang digunakan haruslah sesuai dengan tipe data. Tipe data String dalam parameter tidak bisa diubah menjadi integer dengan pengubahan return x pada nilai return di method kedua.

F. Kesimpulan

Dalam beberapa program, variabel harus disimpan dalam method yang berkaitan dengannya atau menggunakan tipe static untuk variabel diluar method agar program dapat dijalankan dengan sempurna. Ada juga program yang memerlukan metode pemanggilan berjenis static-static atau static-nonstatic untuk dapat dijalankan dengan benar. Program-program lainnya menggunakan teknik overlapping dan penggunaan nilai return yang sesuai dengan tipe data. Kesimpulannya, penggunaan tipe data dan jenis metode yang tepat sangat penting dalam pengembangan program agar dapat berjalan dengan baik dan menghasilkan output yang sesuai dengan keinginan.

Ada tiga macam pemanggilan method di bahasa Java, yaitu:

1. Pemanggilan method berjenis Static-Static

Pemanggilan method jenis ini dilakukan dengan memanggil method yang berjenis static dari class yang sama dengan menggunakan nama class dan nama method. Contohnya:
ClassName.methodName();

2. Pemanggilan method berjenis Static-Nonstatic

Pemanggilan method jenis ini dilakukan dengan membuat objek dari class yang berisi method yang ingin dipanggil. Contohnya:

```
ClassName obj = new ClassName();
obj.methodName();
```

3. Pemanggilan method berjenis Nonstatic-Nonstatic

Pemanggilan method jenis ini dilakukan dengan memanggil method yang berjenis non-static dari objek yang dibuat dari class yang sama. Contohnya:

```
ClassName obj = new ClassName();
obj.methodName();
```

Pemanggilan method yang dipilih tergantung pada jenis method dan bagaimana mereka didefinisikan dalam class. Sebagai contoh, jika sebuah method didefinisikan sebagai static, maka pemanggilannya harus menggunakan jenis pemanggilan Static-Static atau Static-Nonstatic. Sedangkan jika sebuah method didefinisikan sebagai non-static, maka pemanggilannya harus menggunakan jenis pemanggilan Static-Nonstatic atau Nonstatic-Nonstatic. Sedangkan overloading pada Java terjadi ketika terdapat beberapa method dengan nama yang sama di dalam satu class, namun memiliki tipe, jumlah, atau urutan parameter yang berbeda